Sabanci University



Operating Systems

CS 307

Programming Assignment - 1: SUShell Implementation with the Conditional Command

Release Date: 13 March 2026
Deadline: 24 March 2026 23.55

1 Introduction

In the lectures, you have learned that the Command Line Interface (CLI, in short, or shell) is a simple user program that can be implemented using `fork`, `wait`, and `exec` system calls. In the recitation session, you will see that the most fundamental command composition primitive of the UNIX-based operating systems, called *pipes*, can also be implemented inside a simple shell using some file manipulation utilizing POSIX functions `pipe` and `dup`.

In this Programming Assignment (PA), you will be implementing your own shell simulator called SUSHELL . In addition to executing simple CLI commands like `ls`, `wc`, `man` etc., your shell will allow piping commands to run more complex tasks and include a brand new command composition mechanism called Conditional Command which will be referred to as COND-CMD. To make everything a little bit more difficult, file redirection using `'<'` and `'>'` at the end of commands will be possible. Details of the SUSHELL language in terms of both its syntax and semantics are provided in Section 2.

1.1 Motivating Example

Consider the following example that demonstrates the use of multiple SUSHELL features:

```
(cat input.txt ? grep "cs" input.txt | wc -l : echo "error") >
                            output.txt
```

This command consists of four main parts. The first part is a simple command `"cat input.txt"`, which serves as the condition command. If the command produces no output on `stderr` (meaning the file was opened successfully), the then branch is executed. Otherwise, the else branch is executed.

The second part, which is the then branch, consists of `grep "cs" input.txt | wc -l` .The grep command searches for lines containing the word ”cs” in the file, and its output is passed through a pipe to wc -l, which counts the number of matching lines.

The third part, which is the else branch consists of `echo "error"` . It prints the word error indicating that the first part has produced an error.

The last part, `"> output.txt"`, redirects the output that is generated by either the then part or the else part to the file `output.txt` instead of the console.

As can be seen from this example, commands `SUShell` will understand and execute are complex. We strongly recommend you read the whole document at least once before writing any code.

Moreover, we suggest implementing SUShell incrementally by adding features one-by-one according to steps described in Section 6. Even if you cannot implement all functionality, you can get partial points from basic constructs.

2 SUShell Syntax & Semantics

This section formally defines the structure and interpretation of commands accepted by the SUShell . Each subsection introduces a specific feature of SUShell (basic commands, pipes, CondCmd structure, and file redirections), accompanied by the corresponding grammar fragments and illustrative examples.

2.1 Basic Commands

A **basic command** in SUShell is any executable program or binary that can also be run in a regular Unix shell. Commands such as `ls`, `wc`, `man`, and `grep` are all valid in SUShell , as well as binaries specified by their absolute or relative file paths (e.g., `/bin/cat` or `./program`). Any number of options, flags, and arguments can be supplied to these commands.

Examples.

- `ls -l`
- `wc -l input.txt`
- `grep "error" logfile.txt`
- `/bin/cat data.txt`

Every command in SUShell is executed using the system call `execvp()`, which takes the command name and its arguments as an array of C strings.

Obtaining the input array by parsing (processing) an arbitrarily long and complex command can be difficult. Hence, we provide an API method that can do this job for you. The details of this method and how to use it in will be explained in Section 3.1.

2.2 Piping Commands

Pipes are one of the most fundamental composition mechanisms in Unix-based systems. They allow chaining multiple commands so that the **output of one command becomes the input of the next**. Internally, this behavior is implemented through the `pipe()` and `dup2()` system calls, which establish a unidirectional data stream between processes.

The partial grammar that enables piping in SUShell is:

$$Pipe \rightarrow CMD\ \ PipeTail$$
$$PipeTail \rightarrow \texttt{"|"}\ \ CMD\ \ PipeTail\ \ |\ \ \epsilon$$

This means that one or more commands can be combined using the pipe operator `|`. For example, the command `cmd1 | cmd2 | cmd3` executes three processes concurrently, connecting their input and output streams sequentially.

Examples.

- `ls | wc -l`
  This command lists the files in the current directory using `ls`, then sends the list to the next command through a pipe. The command `wc -l` counts the number of lines it receives as input. Therefore, the final output is the total number of files and directories in the current working directory.

- `cat file.txt | grep "error" | wc -l`
  In this example, the contents of `file.txt` are printed by `cat` and passed through two pipes. The first pipe sends the text to `grep "error"`, which filters and outputs only the lines containing the word `"error"`. The second pipe sends those filtered lines to `wc -l`, which counts them. As a result, the final output represents the number of lines in `file.txt` that contain the word `"error"`.

2.3 CondCmd Structure

The CondCmd is the extension introduced by SUShell , designed to extend the standard Unix piping mechanism. In simple terms, a conditional command allows the shell to choose between two different commands depending on whether a preceding command succeeds or fails.

Formally, the syntax of a CondCmd is defined as:

$$\begin{gathered} \mathit{Cmd} \rightarrow \texttt{"("}\ \mathit{Cmd}\ \texttt{"?"}\ \mathit{Cmd}\ \texttt{":"}\ \mathit{Cmd}\ \texttt{")"} \mid \mathit{Pipe} \\ \mathit{Pipe} \rightarrow \mathit{CMD}\ \mathit{PipeTail} \\ \mathit{PipeTail} \rightarrow \texttt{"|"}\ \mathit{CMD}\ \mathit{PipeTail} \mid \epsilon \end{gathered}$$

where:

- `(` and `)` enclose the ternary expression,
- `?` separates the *condition* from the two branches,
- `:` separates the *then_branch* from the *else_branch*,

At runtime, a conditional command has the following structure:

$$\texttt{(}\mathit{condition}\ \texttt{?}\ \mathit{then_branch}\ \texttt{:}\ \mathit{else_branch}\texttt{)}$$

The expression inside the parentheses consists of three components:

- *condition*: the command block used to determine whether the condition succeeds or fails,
- *then_branch*: the command block executed if the condition succeeds,
- *else_branch*: the command block executed if the condition fails.

Each of these blocks represents a complete command according to the grammar. A block may therefore consist of either a single command or a pipeline of commands connected using the pipe operator `|`.

When a conditional command is executed, SUShell first runs the *condition*. The shell monitors the command's error output (stderr) to determine whether the command succeeded. If the condition command produces no output on stderr, the command in the *then_branch* is executed. If the condition

command produces any output on stderr, the command in the *else_branch* is executed instead.

The commands in the *then_branch* and *else_branch* are executed independently and are not pipelined with the *condition*. The output produced by the *condition* is not forwarded as input to the subsequent commands. The standard output (stdout) of the condition command behaves normally and is printed to the shell output unless it is redirected explicitly.

Conditional commands may also be nested recursively, since each block is itself defined as a full command. For example, a conditional command may appear inside another conditional command.

Examples.

- `(grep "cs" courses.txt ? echo "found" : echo "not found")` : If the file courses.txt exists and the grep command runs successfully without producing output on stderr, the command echo "found" is executed. Otherwise, the command echo "not found" is executed.
- `(touch test.txt ? (cat test.txt ? echo "file readable" : echo "read error") : echo "creation failed")` : In this example, the shell first executes touch test.txt. If the file is created successfully, the nested conditional is evaluated. The nested conditional runs cat test.txt; if the file can be read successfully, the command echo "file readable" is executed, otherwise the command echo "read error" is executed. If the first command fails, the command echo "creation failed" is executed instead.

2.4 File Redirection

SUShell supports input and output redirection at the end of a command line. This feature allows the user to replace the standard input or output streams of a command with files. The input redirection operator `<` causes a command to read from a file. In SUShell, this input is provided to the command that is executed first, which is the condition command. The output redirection operator `>` causes it to write to a file. In SUShell, the redirected output corresponds to the final result of the conditional command, meaning that the output produced by whichever branch is executed (either the *then_branch* or the *else_branch*) is written to the file.

The grammar for redirection is:

$$
\begin{aligned}
\textit{OptRedir} &\rightarrow \textit{Redir} \mid \epsilon \\
\textit{Redir} &\rightarrow \texttt{">"}\ \textit{FNAME}\ \textit{OptIn} \mid \texttt{"<"}\ \textit{FNAME}\ \textit{OptOut} \\
\textit{OptIn} &\rightarrow \texttt{"<"}\ \textit{FNAME} \mid \epsilon \\
\textit{OptOut} &\rightarrow \texttt{">"}\ \textit{FNAME} \mid \epsilon
\end{aligned}
$$

This means that, redirection can include only one input redirection; or one output redirection; or both of them (order of input and output redirection doesn't matter); or as stated in the grammar, none of them.

Examples.

- `cat < input.txt`
- `wc -l > result.txt`
- `grep error < logfile.txt > errors.txt`
- `grep error > logfile.txt < errors.txt`

Each redirection modifies the file descriptors of the command before execution:

- Input files (`<`) are opened in read-only mode and attached to `STDIN`.
- Output files (`>`) are opened with the flags `O_WRONLY | O_TRUNC | O_CREAT` to enable writing, truncate existing contents, or create the file if it does not exist, and are then attached to `STDOUT`.

2.5 CLI Command Grammar and Token Definitions

In this assignment, we define a simple **Command-Line Interface (CLI) language grammar** that describes how a command can be written, combined, and executed in a shell. A **grammar** is a set of rules that formally specifies how valid commands are constructed. It provides a precise and unambiguous description of what forms of input our CLI simulator should accept.

Each rule in a grammar defines how smaller pieces (such as words or symbols) can be combined to form larger, valid structures just as the grammar of a natural language defines how words form sentences. In this context, the smallest building blocks are called **terminals** (for example, command names, filenames, and symbols like `|` or `>`), and the abstract combinations of these terminals are called **nonterminals** (for example, *Cmd* or *Pipe*). Together, they define a **Context-Free Grammar (CFG)** that can be used by a parser to check whether a given command line follows the correct structure.

The grammar described in Figure 1 provides the syntax of our CLI command language.

The grammar starts with a **Line**, which represents a single line of input that the shell can understand. A line can either be:

- the special quit command `exit`, which tells the shell to exit, or
- a normal command structure called **Cmd**, that may optionally include input or output redirections.

A **command (Cmd)** represents one executable unit. Commands can either be simple pipelines or conditional commands. Several commands can be joined together with the pipe symbol (`|`) to form a **pipeline (Pipe)**, meaning the output of one command becomes the input of the next, just like in a normal Linux shell.

In addition to pipelines, the shell supports a conditional command structure that behaves similarly to a ternary operator in programming languages. A conditional command has the form:

$$(Cmd\ ?\ Cmd\ :\ Cmd)$$

Each of these components can itself be a full command, which means that conditional commands can be nested recursively and may contain pipelines. This means the grammar supports examples such as:

- `(cat file.txt ? echo "success" : echo "failure")` : print `"success"` if `cat` does not produce an error, otherwise print `"failure"`,
- `(cat file.txt ? grep hello | wc -l : echo "file error")` : if the file can be read successfully, count the number of lines containing `hello`; otherwise print an error message,

Start:

$$
\begin{aligned}
\mathit{Line} &\rightarrow \mathit{Quit} \mid \mathit{Cmd}\ \mathit{OptRedir} \\
\mathit{Quit} &\rightarrow \texttt{"exit"} \\
\mathit{Cmd} &\rightarrow \texttt{"("}\ \mathit{Cmd}\ \texttt{"?"}\ \mathit{Cmd}\ \texttt{":"}\ \mathit{Cmd}\ \texttt{")"} \mid \mathit{Pipe} \\
\mathit{Pipe} &\rightarrow \mathit{CMD}\ \mathit{PipeTail} \\
\mathit{PipeTail} &\rightarrow \texttt{"|"}\ \mathit{CMD}\ \mathit{PipeTail} \mid \epsilon \\
\mathit{OptRedir} &\rightarrow \mathit{Redir} \mid \epsilon \\
\mathit{Redir} &\rightarrow \texttt{">"}\ \mathit{FNAME}\ \mathit{OptIn} \mid \texttt{"<"}\ \mathit{FNAME}\ \mathit{OptOut} \\
\mathit{OptIn} &\rightarrow \texttt{"<"}\ \mathit{FNAME} \mid \epsilon \\
\mathit{OptOut} &\rightarrow \texttt{">"}\ \mathit{FNAME} \mid \epsilon
\end{aligned}
$$

Terminals:

CMD : A single command token (later split into argv), matches $[0-9A-Za-z,.'"-]+$

FNAME : $[A-Za-z_][A-Za-z0-9_.]*$

`"|"` : pipe operator `"("")"` : grouping parentheses

`"?"` : conditional operator `":"` : else separator

`"<"` : stdin redirection `">"` : stdout redirection

`"exit"` : quit shell

ϵ : empty

Figure 1: Complete grammar for the SUShell commands.

- `(which gcc ? (cat Readme.txt ? echo "ready" : echo "readme missing") : echo "compiler not installed")`: This command first runs which gcc. If the gcc compiler exists on the system and the command runs successfully, the nested conditional is evaluated. The nested conditional runs cat Readme.txt. If this command runs successfully, the command echo "ready" is executed; otherwise, echo "readme missing" is executed. If which gcc fails (meaning the compiler is not installed), the nested conditional is skipped and the command echo "compiler not installed" is executed instead.

At the end of any command line, there can be **optional redirections (Redir)**, these specify which files to read from or write to. For instance:

$$< \textit{ infile } > \textit{ outfile}$$

means take input from *infile* and write the final output to *outfile*. The grammar also allows the reverse order (`> outfile < infile`), since both are logically equivalent.

Finally, the smallest elements, called **terminals**, are the actual symbols typed by the user: command names, filenames, digits, and special characters such as `|`, `<`, `>`, `( )`, and `_`.

Together, these terminals and the grammar rules define the complete syntax of the command language supported by our shell.

3 C Implementation

The first and most critical task for any shell is to understand the command a user has typed. This process of taking a raw string of text and converting it into a structured, meaningful representation is called **parsing**. This section details the parser library provided for your assignment, explaining its structure, the data it produces, and the specific API functions you must use to interact with it.

Parsing is the process of analyzing a string of symbols to determine its grammatical structure. The parser uses a set of rules (a **Context-Free Grammar**) to break the input down. The result is an **Abstract Syntax Tree (AST)**, a tree-like data structure that represents the command's logical structure. For example, a command containing pipelines or conditional expressions is represented as a hierarchy of nodes describing how the command

should be executed. A command like `(cat "file.txt" | grep "hello" ? echo "found" : echo "notfound")` is transformed into an AST like the one shown in Figure 2.

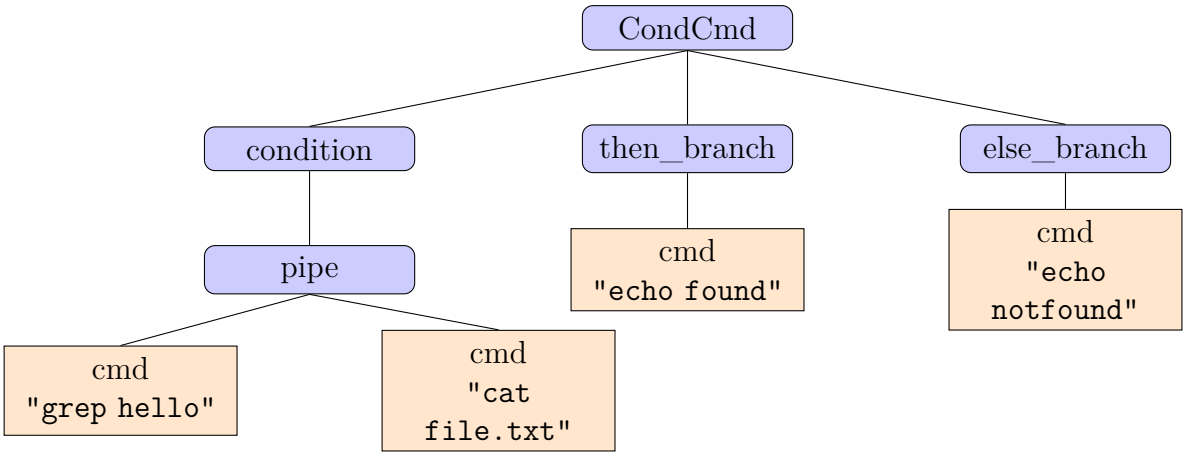


Figure 2: AST representation of the command `(cat file.txt | grep hello ? echo found : echo notfound)`.

The parser library we provide is designed with a clear, two-part structure:

1. A **Complex Command Parser**: The main component reads the input line and constructs an Abstract Syntax Tree (AST) representing the command structure. The tree captures the relationships between commands, including pipelines and conditional commands. Each node in the tree represents either a pipeline of commands or a conditional command with condition, then, and else branches. The parser also identifies input and output redirection files if they are present.

2. A **String-to-Argument-Vector Converter**: A component that takes basic command strings produced by the main parser and converts them into string array format required by `execvp`.

You will **not** write any parsing logic yourself. Instead, you will use the specific API functions described below.

3.1 Parsing Commands

3.1.1 Internal Representation: The `sparser_t` Struct

The `sparser_t` struct is the core of the parser engine. It holds the definitions of the grammar used to understand your shell's command language. When you initialize the parser with `initParser`, this structure is populated with the parser definitions required to analyze command lines. You will **never** directly read from or write to this struct; you will only pass a pointer to it as an argument to the API functions. It is an internal, "black box" component.

3.1.2 The Main API and `compiledCmd` Struct

Your entire interaction with the complex parser will be through the `compileCommand` function and the `compiledCmd` struct it populates.

The `compiledCmd` structure represents the parsed command and contains the root of the command tree together with possible input and output redirections.

The `TopLevel` struct is the clean, final output of the parser and the primary data structure you will work with and its definition is described below:

Listing 1: CmdVec, CmdNode, CmdType, compiledCmd definitions from parser.h

```
typedef struct {
    char ***argvs;
    size_t n;
} CmdVec;

typedef enum {
    CMD_PIPE,
    CMD_CONDITIONAL
} CmdType;

typedef struct CmdNode CmdNode;
struct CmdNode {
    CmdType type;
    union {
        CmdVec pipe;
```

```
        struct {
            CmdNode *cond;
            CmdNode *thenBranch;
            CmdNode *elseBranch;
        } conditional;
    } data;
};

typedef struct {
    CmdNode *root;
    char *inFile;
    char *outFile;
    int isQuit;
} compiledCmd;
```

The parser converts the input command line into a tree representation of the command. The `compiledCmd` structure stores the result of this parsing process.

- **root** : Points to the root node of the command tree. Each node in the tree represents either a pipeline of commands or a conditional command.
- **inFile** : Stores the name of the input redirection file if the command contains `< filename`. Otherwise this field is `NULL`.
- **outFile** : Stores the name of the output redirection file if the command contains `> filename`. Otherwise this field is `NULL`.
- **isQuit** : Indicates whether the user entered the `exit` command. If this value is non-zero, the shell should terminate.

Each node in the command tree is represented by a `CmdNode`. Two types of nodes exist:

- **Pipeline Nodes** : A pipeline node represents one or more commands connected with the pipe operator |. The commands in the pipeline are stored in a `CmdVec` structure. where each command is stored in the format required by execvp.

- **Conditional Nodes** : A conditional node represents a command of the form `( Cmd ? Cmd : Cmd )`. Each conditional node contains three child commands: cond, thenBranch, elseBranch. Each of these fields is itself another CmdNode, allowing conditional commands to be nested arbitrarily.

`compileCommand` is the main method that transforms the line represented by `sparser_t` instance `sp` into the `compiledCmd C` as described below.

```
int compileCommand(sparser_t *sp, char *input, compiledCmd* C)
```

- **Functionality**: This is the main entry point to the parser. It takes the initialized parser engine (`sp`), the user's raw input string (`input`), and a pointer to a `compiledCmd` struct (`C`) that you want it to fill. The function uses the grammar in `sparser_t` to build an AST, traverses that tree, and extracts all relevant information into the simple, easy-to-use `compiledCmd` struct.
- **Return Value**: An `int`. A return value of **1** indicates a successful parse. A return value of **0** indicates a syntax error.
- **Example**: If the input string is `"(cat file.txt | grep hello ? echo found : echo notfound)"`, after calling this function, the `compiledCmd` struct you passed in will be populated as follows:
  - `root`: a conditional command node whose fields are
    - `cond`: a pipeline node containing a `CmdVec` with
      - `argvs`: `{{"cat", "file.txt", NULL}, {"grep", "hello", NULL}}`
      - `n`: 2
    - `thenBranch`: a pipeline node containing a `CmdVec` with
      - `argvs`: `{{"echo", "found", NULL}}`
      - `n`: 1
    - `elseBranch`: a pipeline node containing a `CmdVec` with
      - `argvs`: `{{"echo", "notfound", NULL}}`
      - `n`: 1
  - `inFile`: `NULL`
  - `outFile`: `NULL`
  - `isQuit`: an int, `0`

3.1.3 Remaining API Methods

`int initParser(sparser_t *p)`

- **Functionality**: Initializes the parser engine. It constructs the parser objects used to recognize the command grammar and prepares the `sparser_t` structure for use. This function **must** be called once at the very beginning of your program.

`void freeParser(sparser_t *p)`

- **Functionality**: Cleans up all resources used by the parser engine. This function **must** be called once at the very end of your program.

`void freeCompiledCmd(compiledCmd *c)`

- **Functionality**: Frees all memory associated with the contents of a compiledCmd structure, including the command tree, internal argument vectors, and any redirection filenames. This **must** be called after you have finished executing a command.

`void printCompiledCmd(compiledCmd *c)`

- **Functionality**: Prints the `compiledCmd` struct. You can use this function to understand the structure of compiledCmd and debug easily. You will not use this function in the final implementation of the homework.

`void initCompiledCmd(compiledCmd *c)`

- **Functionality**: This function initializes a compiledCmd structure. It is called internally by compileCommand before populating the structure. You do not need to call this function directly.

3.2 SUShell Program Flow

Below is a general description of the program flow.

Parser setup. Before entering the main loop, the parser must be initialized using: `int initParser(sparser_t *p)`. If initialization fails, the program should terminate.

Interactive loop. Since SUShell is designed as an interactive shell, it continuously waits for user input. After the execution of the input command, shell waits for the next input. At each iteration, the shell prints its prompt such as `SUShell$` and flushes it. Then it reads the command using `getline`. If `getline` returns -1 (for instance, when the user presses `Ctrl+D`), the loop breaks and the program performs cleanup before exiting.

Parsing. The input line is passed to the parser using `compileCommand`. If the parsing fails, the shell prints `parse error`, frees the memory allocated for `C` using `freeCompiledCmd`, and continues to the next prompt without terminating.

File redirection. Before executing, the program checks for any input or output redirection operators (`<`, `>`). If such redirections exist, it opens the specified files. If a file cannot be opened, the shell cleans up and skips to the next iteration. If no files are provided, `STDIN` and `STDOUT` are used by default.

Execution. After the setup phase, you are expected to implement the execution algorithm that simulates how a shell runs commands. Each basic command must be run in a new process, and if two basic commands are connected through a pipe, you must arrange the file descriptors of processes corresponding to these commands such that one's output flows into the next one as its input. The shell should print the next prompt only after the entire command has completed execution.

Cleanup and resource management. After every command, the shell must free all allocated memory and close any open file descriptors. For this purpose, the helper function `freeCompiledCmd` should be used and file descriptors should be closed to ensure that no memory or file descriptors leak between commands.

Shutdown. When the user enters the quit command (`exit`) or the end-of-file signal is received, the shell terminates the loop, frees all remaining parser objects using `freeParser`, and exits by returning `0`. If the user enters the command (`exit`), terminate by printing `"Exiting shell..."` followed by a newline character.

4 Report Contents

In your report, you must answer the following questions explicitly, in **separate** sections.

- Explain your process hierarchy. During a pipe command execution, in which order does the parent (shell) process create command processes? Can they run concurrently? To what extent? Please, explain your answers over a figure.
- Please, describe the pipe structure you construct between two piped processes. How many pipes do you create per piped command? How do you make sure that the latter command gets the correct input from the former one? How do you prevent deadlocks and ensure that unused pipe file descriptors are properly closed in each process?
- Explain how your shell executes a conditional command (cond ? then_branch : else_branch). How do you determine whether cond succeeds or fails, and how do you execute nested conditional commands recursively?
- Explain how your implementation handles input and output redirection. How do you pass the input file descriptor to the condition? How do you ensure the output file descriptor is applied to the correct branch and not to the condition itself?

Please note that if your answers to the report do not match your code, you will not get any credits from your report.

5 Submission Guidelines

For this homework, you are expected to submit two files. The first is a pdf that contains your report. The name of this file must be *"report.pdf"*. The second file you will submit is the code part of your assignment. The name of this file must be *"shell.c"* Below you can see the content of your files.

- shell.c: This file will contain your SUShell implementation.
- report.pdf: Your report that explains your code and answer the questions. Please note that all the reports are read carefully to understand your program in a better sense. In your report, you are expected to

make the instructor understand such details as : What your program does, how your functions work (if there is any), what is the role of pipe on that function etc.

During the submission of this homework, you will see two different sections on SUCourse. For this assignment you are expected to submit your files separately. You should NOT zip any of your files. While you are submitting your homework, please submit your report to "PA1 – REPORT Submission" and your code to "PA1 – CODE Submission". SUCourse will not except files in another format, so please be careful. If your submission does not fit to the format specified above, your grade may be penalized up to 10 points.

6 Grading

Dear students, your first programming assignment will be graded using automated test cases. Please keep in mind that even slight discrepancies on output will make you lose points.

- Compilation (10 pts): Your program compiles, runs and terminates without an error.
- Basic command execution (10 pts)
- Basic command with input or output redirection (10 pts)
- Piped command (10 pts)
- Basic CondCmd (20 pts): We will check both with and without redirection.
- Nested CondCmd (20 pts): Again, file redirection can be used for these commands.
- Report (20 pts): Your report has to meet the requirements described in Section 4 and be consistent with your implementation.

Each item is a precondition for the next one except the last item. It means that, if you do not get at least half of the points for an item, we will not test your programs for the next items.

Note that grading will be done automatically. Your shell's output will be redirected to a file and compared to the expected answer. If your shell name or whitespace characters differ, they will be considered incorrect.

Also note that commands containing characters not defined by the grammar, or commands that do not conform to the grammar, will result in a "parse error". Any cases where the parser produces a parse error will not be considered during grading.

7 Penalties

Dear Students, we would like to mark down some common mistakes for this Programming Assignment that will lead to point reduction.

- If your program is not a multi-process one, then you get 0.
- You have to use fork-exec couple for creating new process. Using other system calls like `"system"` is prohibited. If you do not obey this rule, you will get a 50 points penalty.
- Incorrect name usage (-5 pts): Please note that the c file you will submit must be named as `"shell.c"`.
- If your report is not consistent with your implementation, you will not get any points for your report and lose 20 overall points.

8 Sample Runs

8.1 Makefile, Compilation & Testing

Here is a one-time, detailed explanation of how to compile C files in a Linux terminal, in case some of you haven't done this before. We will cover what compilation is, how to do it manually, and how to use a `makefile` (the recommended way) to automate the process.

Briefly, compilation is the process of using a program (called a compiler, in our case gcc) to translate your human-readable C code (.c files) into a single, machine-readable executable file that the operating system can run.

First, you must add the following line to the top of your `shell.c` file, along with your other includes (like `<stdio.h>, <unistd.h>,` etc.). This line gives your `shell.c` file access to the functions and structures defined in the parser.

```
#include "parser.h"
```

To create your shell, you need to compile all three source code files (`mpc.c, shell.c, parser.c`) together into one executable. You can do this by running the following command on your terminal. To be able to run this command, all those three files must be under the same directory.

```
> gcc -o shell shell.c parser.c mpc.c -Wall -Wextra
```

By running this command, you can see the executable named `shell`, under the same directory with your source codes. After you generate the executable you can run it and start your testing with this command.

```
> ./shell
```

Instead of performing all these steps manually, you can use the provided Makefile to easily compile your source code. To do this, place all the files in the same directory. Remember, all your files must be named correctly for the Makefile to work properly. Then, simply run the following command:

```
> make shell
```

This will basically do the same operation with manual command and generate an executable file.

8.2 Sample Runs

Sample Run 1

The first test demonstrates that the shell can execute basic commands just like a regular Unix shell. These examples contain single commands without piping, conditional structure, or file redirection.

Command: `./shell`

Terminal Output:

```
SUShell$ echo "hello cs307"
hello cs307
SUShell$ seq 1 3
1
2
3
SUShell$ uname -s
Linux
SUShell$ /bin/echo basic command works
basic command works
SUShell$ exit
Exiting shell...
```

Sample Run 2

This test demonstrates the execution of piped commands. In these examples, the output of one command becomes the input of the next one through pipes created by the shell.

Command: `./shell`

Terminal Output:

```
SUShell$ echo hello | wc -c
```

```
6
SUShell$ seq 1 5 | tail -n 3 | wc -l
3
SUShell$ cat courses.txt | grep cs | wc -l
2
SUShell$ echo "abcdef" | rev
fedcba
SUShell$ exit
Exiting shell...
```

Sample Run 3

Like a regular shell, SUShell supports input and output redirections. The following examples show commands that read from a file using `<`, write to a file using `>`, or do both together. The file `input1.txt` should be prepared before running these commands.

Command: `./shell`

input1.txt:

```
alpha
beta
gamma
```

Terminal Output:

```
SUShell$ cat < input1.txt
alpha
beta
gamma
SUShell$ wc -l < input1.txt
3
SUShell$ echo "written to file" > output1.txt
SUShell$ wc -l < input1.txt > count.txt
SUShell$ exit
Exiting shell...
```

output1.txt:

```
written to file
```

count.txt:

```
3
```

Sample Run 4

This test demonstrates the basic CondCmd structure of SUShell. The shell first executes the condition command and monitors its `stderr`. If the condition produces no output on `stderr`, the then-branch is executed; otherwise, the else-branch is executed.

Command: `./shell`

courses.txt:

```
cs
math
cs
```

Terminal Output:

```
SUShell$ (cat courses.txt ? echo "file opened" : echo "
   error")
cs
math
cs
file opened
SUShell$ (cat missing.txt ? echo "file opened" : echo "
   error")
cat: missing.txt: No such file or directory
error
SUShell$ (grep "cs" courses.txt ? echo "found" : echo "
   not found")
cs
cs
found
SUShell$ exit
Exiting shell...
```

Sample Run 5

The following examples show conditional commands whose branches contain

pipelines. This demonstrates that each branch of a conditional command can itself be a complete pipeline.

Command: `./shell`

courses.txt:

```
cs
math
cs
```

input1.txt:

```
alpha
beta
gamma
```

Terminal Output:

```
SUShell$ (cat courses.txt ? grep cs courses.txt | wc -l :
    echo "error")
cs
math
cs
2
SUShell$ (cat missing.txt ? grep cs courses.txt | wc -l :
    echo "error")
cat: missing.txt: No such file or directory
error
SUShell$ (cat input1.txt ? cat input1.txt | wc -l : echo
   "bad input")
alpha
beta
gamma
3
SUShell$ exit
Exiting shell...
```

Sample Run 6

Nested conditional commands are also supported, because each branch of a conditional command is itself a full command. The following examples demonstrate nested CondCmd evaluation.

Command: `./shell`

courses.txt:

```
cs
math
cs
```

input1.txt:

```
alpha
beta
gamma
```

Terminal Output:

```
SUShell$ (cat courses.txt ? (cat courses.txt ? echo "
   inner ok" : echo "inner fail") : echo "outer fail")
cs
math
cs
cs
math
cs
inner ok
SUShell$ (cat missing.txt ? echo "outer ok" : (cat
   courses.txt ? echo "recovered" : echo "still bad"))
cat: missing.txt: No such file or directory
cs
math
cs
recovered
SUShell$ (cat missing.txt ? echo "outer ok" : (cat
   missing2.txt ? echo "recovered" : echo "final fail"))
cat: missing.txt: No such file or directory
cat: missing2.txt: No such file or directory
final fail
SUShell$ exit
Exiting shell...
```

Sample Run 7

This sample shows the interaction between conditional commands and file redirection. Input redirection is applied to the first executed command, which is the condition command, and output redirection captures the final result produced by the executed branch.

Command: `./shell`

input2.txt:

```
red
blue
red
```

Terminal Output:

```
SUShell$ (cat ? echo "ok" : echo "bad") < input2.txt
red
blue
red
ok
SUShell$ (cat missing.txt ? echo "ok" : echo "bad") >
   output2.txt
cat: missing.txt: No such file or directory
SUShell$ (cat input2.txt ? grep red input2.txt | wc -l :
   echo "error") > output3.txt
red
blue
red
SUShell$ exit
Exiting shell...
```

output2.txt:

```
bad
```

output3.txt:

```
2
```